Functional requirements

STOCK EXCHANGE

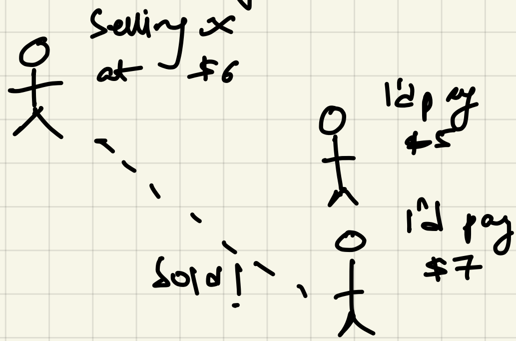
diff from stock brokers!

- Match buyers & sellers of stock!
- ordered set of operat'ns that everybody agrees on
  - eventually!
  - fault tolerant

↓

cancellat'n
order placement
etc.

General algorithm



The limit order book

(matching algo)

Keeping track of bids & asks

Bids

RK: ~~$100~~ 130 x 110 x5
MB: $90 x 10

Asks

C: 120$ x 100
M: 130$ x 10
R: 140$ x 20

one or more trades exec when bid price ≥ ask price

X Trades

✓ 130 x 110 | bid
120 x 100
10 x 10 | sell

Performance is key: xx × 10^6 orders daily.

Low throughputs result in many of these trades to start crawling

↓

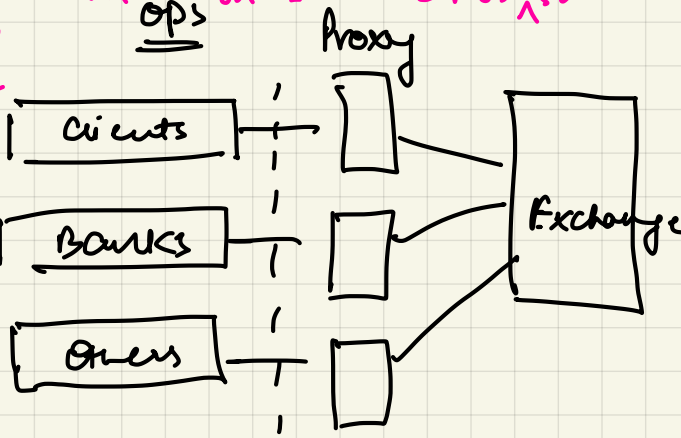
limit I/O as much as possible (n/w & disk ops)

{ Matching bids to asks should be at high throughputs }

Limit locks on I/O.

Deterministic req: S → S' {set of msgs}

1↔1 connection, threads run on same CPUs and n/w ↑, can't multicast to all at once



TCP ✗

UDP multicast ✓
- msg drop possible
- msg rcv out of order

Multicast networking or communication is essential here. ∵ same data needs to be sent to all clients.

connection should be "of air".

Data should reach at any of the clients at same time w minimal latency

I/O is to be avoided here

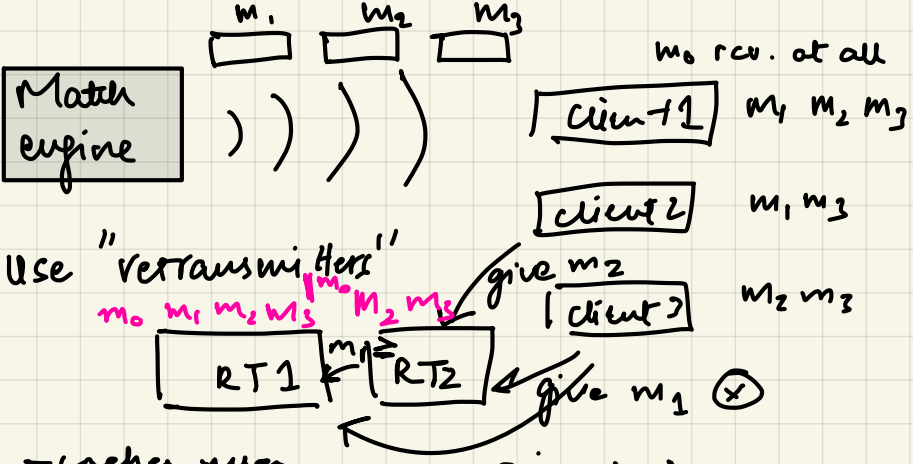
commit logs should not be used.

↓

or can't. Use multicast to drop msgs in commit log (multiple) and clients can read them however they want?

IGMP Layer 2 collects data from stream

PIM Layer 3

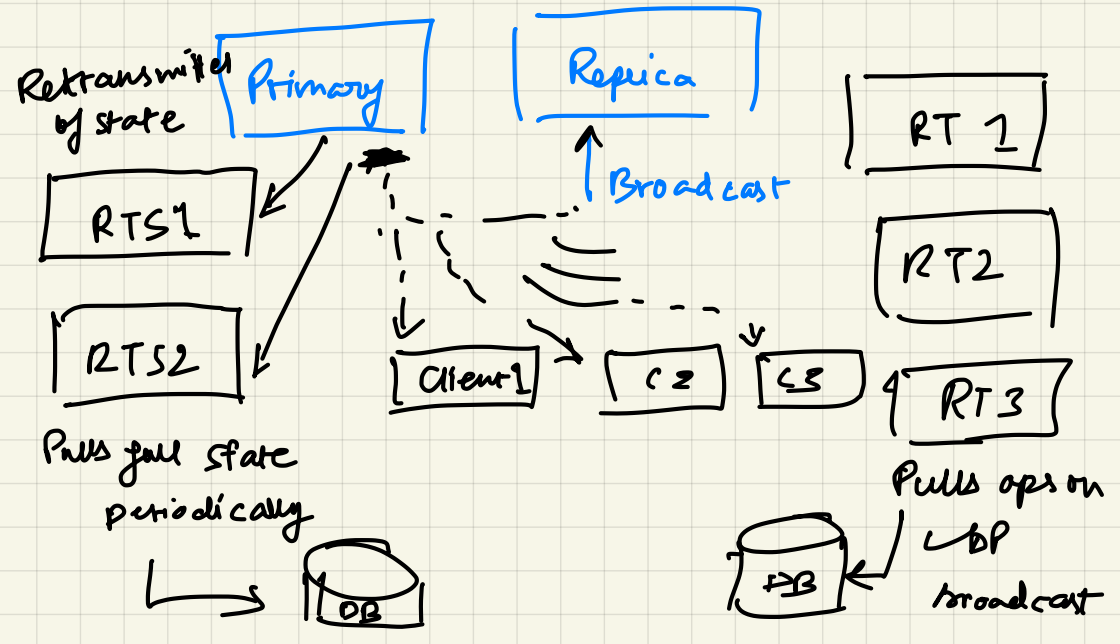


- caches msgs
- a lot of RTs used to reduce PL dropping messages)
- RTs communicate internally as well to sync

fault tolerance via state m/c replicat^n.

- runs in m/m. Backups are essential. everything

Initial state copied to (P)

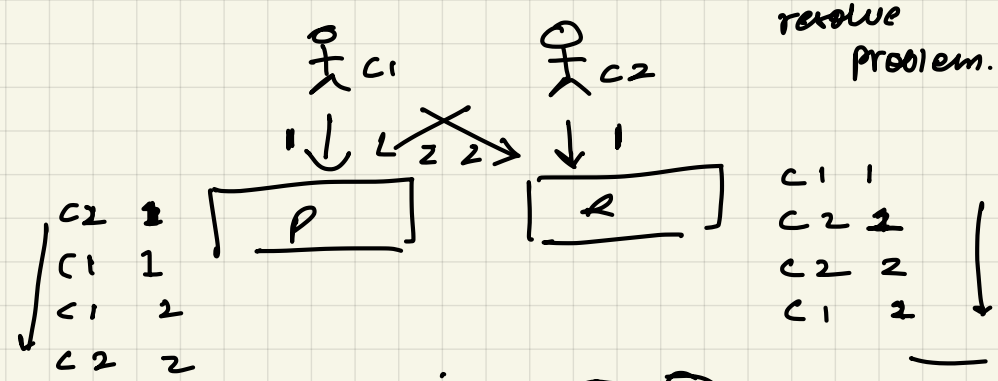


Whenever ① goes down ② takes over. And new ②, ②2 pulls last state from RTSx and replays msgs after it via RTx & to develop fresh state.

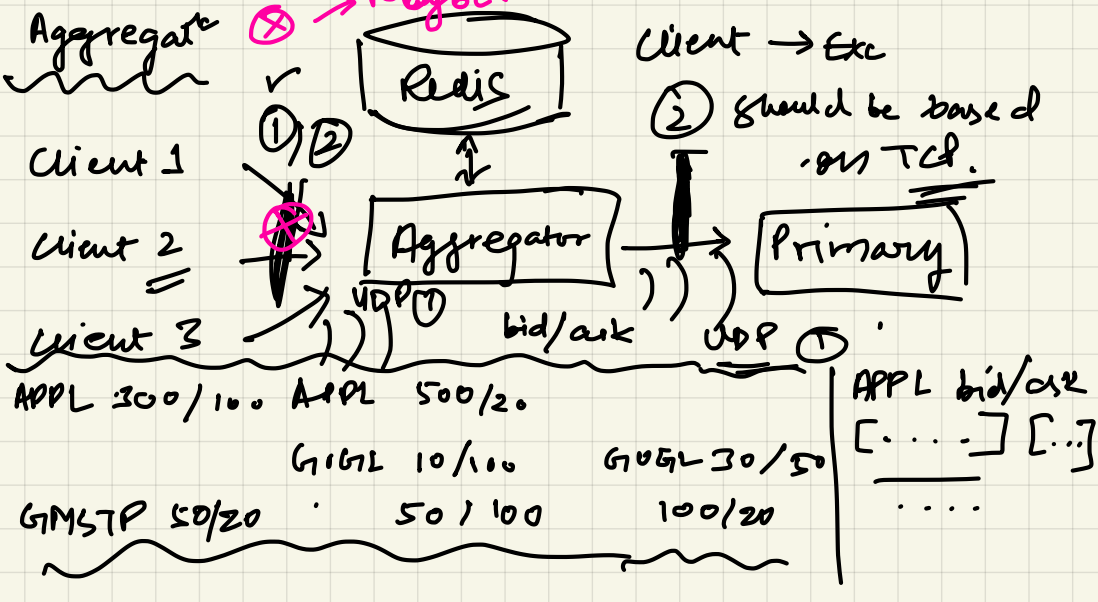
SAGA algo :

UDP broad cast ②

Zookeeper with heartbeats to resolve problem.



order is possibly diff on P & R.



Multicast should be triggers for all transactions even if we're using ① for Ack. and telling everyone about offered trades.

For ① we'll need RT(S) & RT(S').
   /②

Partitioning : we're relying on a single leader, pa..." is essential.

Partitioning on multiple stock collections on a single server is essential. ordering can't be maintained otherwise.

→ I/O would be introduced
  - DB interactions
  - n/w

→ Race condⁿ and order violations possible.

128/256 GB m/c's are needed.

Atomicity on multiple symbols need a slow distributed transactions.

Ordering on matching engine

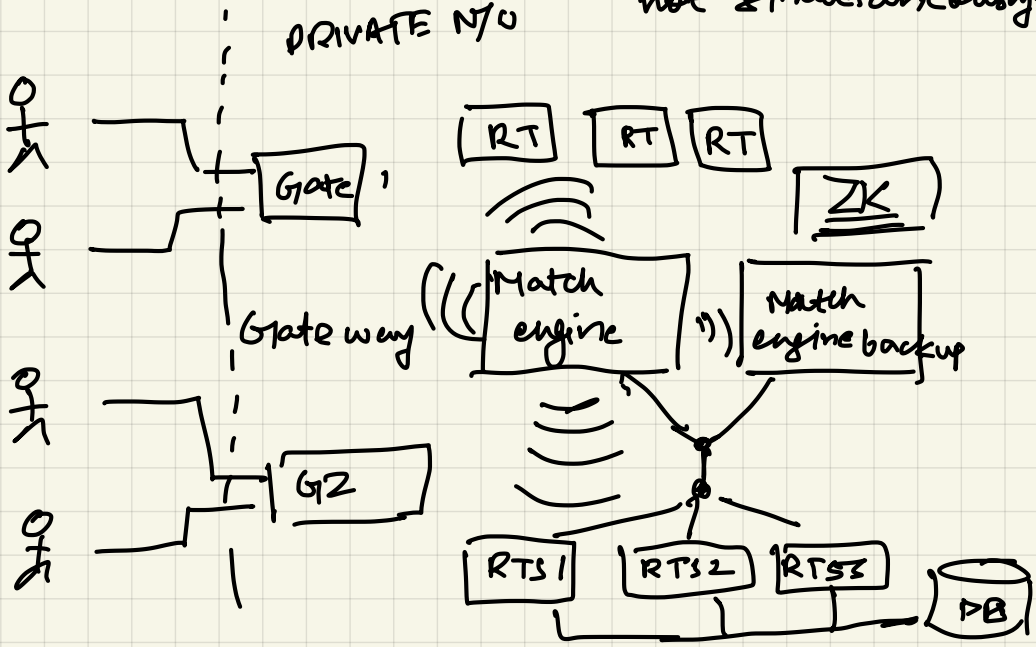
Multi-threaded env is still there.

OP i/p ⇒ grab lock on relevant client/DS

[exec BID AAPL 1000 20 client 1
ASK AAPL 1020 30 C2
BID APPL 10010 C3] id=1001

Lock on C1, C2, C3 reqd. to get a reqd. seq number.

[cancel ASK AAPL 1020 30 C2] id=1002
will be exc~~uted~~ after 1001
or before
not simultaneously.



YOUTUBE SYS DESIGN

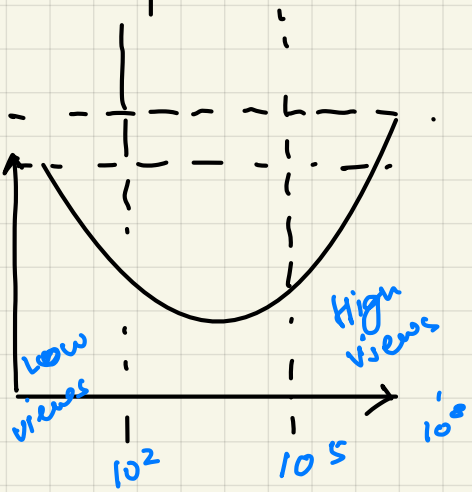
→ upload + download/watch ★
→ Search ★
→ Recommend [a separate topic]
→ Commenting ["]
→ Analytics [a separate topic]

Functionality

Non funct. req.

Scale : 1B users
400 M DAU

Streams / downloads
avg : 10^4 : 1 uploads
(assumed)



week

no. of uploads per day : $10^9 \times 0.2$

1 video = 3GB
↓
1.2GB

$\Rightarrow 1.2\,GB \times 10^9 \times 0.1 \times 60$

$\Rightarrow$